Project 1: Count Down Timer App (Group Project)

Due Date: As per D2L Dropbox

Task: Using the Microcontroller and the driver functions developed so far, you will design a *Count Down Timer program* ("the program") to be displayed on a PC's terminal (e.g., RealTerm) as described in this brief. The program will use the push buttons (PBs), PB1, PB2, and PB3, connected to ports RB7, RA4, and RB4, respectively. The Timer App should operate as follows. You **must** use the UART, Timer, and CN peripheral(s). Two LEDs should also be used, connected as per previous assignments.

Before the timer starts/runs:

User input(s)	Output(s)
While PB1 is pressed (without PB2 or PB3 pressed)	Set the seconds to be counted to. When PB1 is pressed, the seconds count increments by 1 sec from 0 to a maximum of 59, before rolling over back to 0. • While PB1 is held down, the seconds should continue to increment at a pace slow enough for a user to stop at a desired count • After holding down the button for some time and incrementing (for instance, up to the next number ending in 0 or 5), the increment amount should change to go up by 5 while the button is held down (pressed). The exact amount of time the button needs to be held before the increment by 5 begins should be decided by each individual group. The incrementing second count should be displayed on the PC terminal as follows. **`SET 52m : 45s`** This should also clear any ALARM message
While PB2 is pressed (without PB1 or PB3 pressed)	Set the minutes to be counted to. When PB2 is pressed, the minute count increments by 1 minute from 0 to a maximum of 59. • While PB1 is continued to be pressed (held down), the timer should continue to increment. Increments should occur at a pace slow enough for a user to stop at a desired minute count for e.g., 52nd minute shown below. The incrementing minute count should be displayed on the PC terminal as follows. **`SET 52m : 00s`** This should also clear any ALARM message
While PB3 is held down	If PB3 is held down by itself, there should be no change to the current count. Pressing/holding PB1 or PB2 together with PB3 should cause the seconds/minutes to decrement by 1, respectively.
Short press of PB1 and PB2 together	This should start the timer. The timer should start decrementing from the programmed time, with the update shown for each second that passes.
PB1 and PB2 held together	This should reset both minutes and seconds back to 0. (i.e., `00m : 00s`)

Holding down all 3 PBs	While all 3 push buttons are held, the terminal should display your group's information: **`2026 ENSF 460 L01 – Group xx`**
No buttons pressed	Note: While no buttons are pressed, the program should be in a low-power mode as much as possible (i.e., the CPU should be in Idle mode), with no new updates on the terminal/UART.

After starting the timer/while the timer is running:

No buttons pressed	The timer should continue to decrement. Your timer should match real world clock time (i.e., if you start a timer on your phone at the same time as your program, they should reach 0 at the same time). The PC terminal should be updated to display the decremented minutes and/or second every second for example: **`CNT 05m : 08s`** While the timer is counting down, LED1 should blink, 1 second on, 1 second off, until you reach 0:00. When the desired time is reached, the PC terminal should be updated to display the following: **`FIN 00m : 00s – ALARM`** When the timer is done, LED1 should remain solidly on, and LED2 should start to blink rapidly. Note: While no buttons are pressed, the program should be in a low-power mode as much as possible (i.e., the CPU should be in Idle mode), with no new updates on the terminal/UART.
Short presses (< 3 sec) on PB3	A short press of PB3 will pause the timer at its current value. A subsequent press of PB3 should resume the timer.
Long press (> 3sec) on PB3	A long press on PB3 stops the count and resets the timer to 0, and should be shown on the PC terminal: **`CLR 00m : 00s`**

Additional info:

Implement the above controller using the hardware kit and your code, which will be designed using basic C commands, IO(CN), and Timer interrupts, and UART functions. **Use of polling of PORTx or TMRx directly, instead of interrupts, will lose points.** Note, you may use multiple timer peripherals, or the 32-bit timer configuration, at your discretion. You will need to navigate the datasheet by yourselves, accordingly. Your implementation should be as power-efficient as possible. You may choose whichever system clock frequency you wish.

Function names: Students can use any convention when naming functions or organizing code. A state diagram of your design is required as part of your submission. Use microcontroller-specific register and bit names wherever applicable in the state diagram.

Display instructions: All displays on the PC terminal window must be on a single line. Note that display functions carried out at 32 kHz (300 Baud) can affect timer delays. Your code should account for such delays when producing delays specified in the table above.

Interrupts: Interrupt ISR names are provided in the lecture slides. As specified in the lecture, IO (CN interrupts) are triggered on rising and falling edges, and due to any bounce effects of the push buttons that cause several high-to-low and low-to-high fluctuations at the Microcontroller input. Your code may need to filter out any such effects.

Your submission must satisfy the following in addition to achieving all requirements of this project and delivering all deliverables: (i) You do not call functions in an ISR (this includes no UART messages in the ISR), (ii) You do not have infinite loops in an ISR, (iii) You do not add delays to the ISR, (iv) You do not poll the input ports (i.e., your program should not be repeatedly testing (reading) the value of the PORTA or PORTB bits – your program should be designed such that the CPU spends most of its time in Idle()[1] and only "wakes" to deal with something after an interrupt has occurred), (v) Your submission should not have "glitches" (e.g., pushing the buttons in an arbitrary order at arbitrary times should not result in unexpected behavior), (vi) the timing should not be significantly different to a smartphone/desktop timer.

You may submit a project that does not satisfy all requirements, but you will only receive partial credit. For instance, you might first try to implement a basic increment and time count, without pause or multiple buttons.

Proper design practices, as discussed in the lectures, must be followed.

All members of your group must attend and participate in the demo.

If it helps to clarify the functional requirements, the interface of your countdown timer bears a vague resemblance to the time-setting functionality of a basic digital alarm clock.

[1] See *10.2.2* in the PIC24F16KA102 Family Datasheet

Deliverables:

This is a group project. Each group should upload the following to their respective group D2L-Dropbox folder.

1. **Zipped up file of the MPLAB project**. MPLAB projects can be zipped up by right-clicking on the project and selecting "package." The zipped project is saved in the same project folder created by the user. Ensure your driver code is properly commented. **This must be submitted before the demos and this version will be used for marking.** It is your responsibility to upload the correct files.

2. **A single PDF document** showing at least the following:
    a. Names and UCIDs of all students in the group at the top of the document
    b. **A State Machine diagram** showing the design of your program. Use microcontroller-specific register and bit names, and code-specific function names/labels wherever applicable in the state diagram.
    c. A report with a description of all important elements of your implementation (supported by screenshots/snippets, or other diagrams and flow charts, as appropriate). Remember to include details about how you implemented elements, such as determining if a long or short press has occurred.
        - You must also identify and explain potential "corner cases" that you needed to handle. For example, what happens if a user tries to start the timer with a time of 00:00? As per the course outline, when in doubt, you should seek clarification from the instructor.

3. **A completed group work form. This must be endorsed (signed) by all group members. Note: Individual marks might be adjusted to account for imbalances in the group participation. All students must have made technical contributions to the project.**

4. **Perform a live demo of your project to members of the teaching team in the lab or during the teaching times of the week, starting October 26.**
    a. To perform your demo, you will first download your submission from D2L **in front of** the assessor.
    b. You will have **12 minutes** to demonstrate your project, which includes the time to download and re-import your project to MPLAB X IDE to run on your microcontroller.
    c. Demo of the code and hardware operation showing all states, preferably with reference to your state machine. **To verify the timing of your count, have your microcontroller app running adjacent to your smartphone's countdown timer app**. Each group member should be prepared to answer questions related to the working of their app project.
    d. Your group will be scheduled a time during your timetabled lab session or lecture times. A schedule will be published closer to the time.

Note: You are not allowed to share laptops/code/hardware between groups. Plagiarism of code or report is not allowed. Please monitor your email inbox and D2L in case there are any changes to the deliverables, clarifications to the project, or other communication from your instructor, as the project's requirements may be subject to change.

Project Rubric

	Fails to meet expectations	Minimally meets expectations	Adequately Meets Expectations	Exceeds Expectations
Peripheral Configuration and Use Program Logic	None of the peripherals and states working correctly Does not provide evidence of appropriate program flow **0 points**	Some of the peripherals and states working correctly to implement the functionality in the brief Provides evidence of attempting to use C control statements but has some errors or does not cover all scenarios in the brief **12 points**	Most of the peripherals and states working correctly to implement the functionality in the brief Provides evidence of attempting to use C control statements to cover most but not all scenarios in the brief or has some errors **16 points**	All peripherals and states correctly working to implement the functionality in the brief Provides evidence of appropriate C control statements and implements all scenarios correctly as in the brief and other scenarios in a justified manner **20 points**
Response to task	Does not address all aspects of the brief. **0 points**	Submission addresses most, but not all aspects of the brief. Several elements of the submission are unclear, vague, or inaccurate. **3 points**	Submission addresses most, but not all aspects of the brief. Few elements of the submission are unclear, vague, or inaccurate. **4 points**	Convincingly addresses all aspects of the brief in an accurate, clear, and concise manner. **5 points**
Efficiency and usability	Does not provide evidence of using power-efficient features (e.g., Idle) or usability **0 points**	Provides some limited evidence of using power-efficient features (e.g., Idle, clock switching) and usability **3 points**	Provides some evidence of using power-efficient features (e.g., Idle, clock switching) and usability (robustness of changing timer value, resetting the timer, etc.) **4 points**	Provides strong evidence of using power-efficient features **appropriately** (e.g., Idle, clock switching) and usability (e.g., robustness of changing timer value, resetting the timer, etc.) **5 points**
Code Comments/ Structure	No evidence of code commenting or reasonable variable names. **0 points**	Some evidence of code comments, but infrequent or incomplete. **3 points**	Evidence that most elements of the code are commented. **4 points**	Evidence that all important elements of the code are commented meaningfully. **5 points**
State Machine Diagram and Description	No state machine diagram uploaded or unclear diagram. **0 points**	State machine design appears to be an FSM, but several states missing or incorrect. **3 points**	State machine design appears to be an FSM, but may have some misalignment with the implementation. **4 points**	Proper state machine showing all states and using Microcontroller and code specific register/flag/function names. Implementation clearly matches design. **5 points**
Report/ Documentation	No report provided or report with vague details as to the logic of the design and implementation. **0 points**	Basic report provided that outlines at least some of the important elements of the design and implementation. May have some errors of expression (e.g., typographical errors, inappropriate tone) **3 points**	Basic report that provides details of some of the important elements of the design and implementation, supported with relevant screenshots/diagrams/snippets of code, as appropriate. May have some errors of expression (e.g., typographical errors, inappropriate tone) **4 points**	Detailed report that provides details of all the important elements of the design and implementation supported with relevant screenshots/diagrams/snippets of code, as appropriate. Professionally presented. **5 points**
Teamwork	Not all team members have a reported contribution to the submission **0 points**	All team members have reported that they have contributed to the submission, but areas of contribution are unclear **3 points**	All team members have reported that they have contributed to the submission, but areas of contribution are a little vague **4 points**	All team members have reported that they have contributed to the submission, with areas of contribution clearly specified and with appropriate levels of balance across team members. **5 points**
				Total (50):